SMART HOME SYSTEM

By: Jasmine Martinez & Ava Marshia

CDA3631.01

Spring 2026

Introduction 2
Materials 2
Methods 2
Initializations 3
Software Design and Coding Method 3
Wiring and Hardware Method 4
Summary of Method 4
Results & Discussion 5
Conclusions 6
Acknowledgements 6

Introduction

The rapid advancement of embedded systems and Internet of Things (IoT) technologies has enabled the development of intelligent home automation solutions that enhance convenience, energy efficiency, and user comfort. A Smart Home Controller integrates multiple devices—such as lighting systems, fans, temperature sensors, and motion detectors—into a centralized system capable of monitoring environmental conditions and executing control actions in real time.

This project focuses on the design and implementation of a Smart Home Controller using a real-time operating system (RTOS), specifically FreeRTOS. By leveraging multitasking capabilities, the system assigns dedicated tasks to each device, enabling concurrent operation and efficient resource management. Communication between tasks is handled through queues or message buffers to ensure reliable data exchange, particularly for sensor readings. Additionally, synchronization mechanisms such as mutexes are utilized to manage shared resources like display interfaces (e.g., LCD or OLED), preventing conflicts and ensuring system stability.

Materials

For our smart home project, we have used a variety of different materials. It is conducted using a STM32 NUCLEO F446RE board for the microcontroller. For temperature input, we used a DHT11 sensor. We decided to use a fan in conjunction with the temperature sensor. This fan will be triggered “on” when the temperature rises to a specific value. The representation of the fan is a servo motor with the fan attachment. One thing that we wanted to include is a type of security system. For this, we decided to choose a PIR sensor that would trigger an LED light upon

detection. The LED represents a house light or strobing light being triggered “on” by detection. We will also be utilizing an LCD display to present the system’s status. Using FreeRTOS, each device will be controlled by a separate task. The sensors will send data to queues, and the display is shared using mutex.

Methods

The Smart Home Controller system was developed using the STM32 NUCLEO-F446RE microcontroller and programmed in C using STM32CubeIDE with HAL and FreeRTOS. The system was designed using a modular, multitasking approach where each major function (sensing, control, display, and user input) operates as an independent real-time task. By making them as independent tasks, it allows for efficient concurrency, responsiveness, and scalability.

Sensor data is collected, packaged into structured messages, and transmitted through a queue to other system components for processing and display.

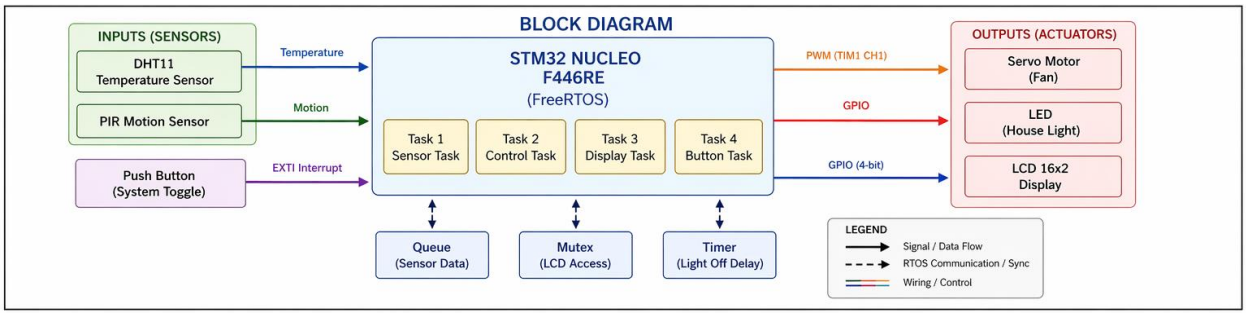


Fig. 2 Block diagram for smart home

Initializations

All peripherals were initialized during system startup using STM32 HAL drivers. The initialization process included configuring GPIO pins, ADC channels, timers, and communication. Output pins were used for the LED, LCD interface, and servo control signals. Input pins were configured with interrupt capability for the push button and the PIR motion sensor. External interrupts were enabled to allow events to be connected at the same time. ADC1 was configured with 12-bit resolution to read temperature sensor values. The conversion result is then translated into a voltage and converted into temperature. TIM1 was configured for PWM output to control the servo motor. The PWM signal is used to set the angular position of the servo, which represents the fan speed. USART2 was configured for serial communication at 115200 baud. FreeRTOS objects were initialized, including a message queue for sensor data transmission, a memory pool for efficient message allocation, a mutex for LCD resource protection, semaphores for button and motion event signaling, and a software timer for timed operations. These components enable safe and efficient inter-task communication and synchronization.

Software Design and Coding Method

The system software was designed using FreeRTOS with four primary tasks: sensor task, control task, display task, and button task. The sensor task reads temperature and motion data. The temperature is received through ADC sampling and filtered for stability, while motion is detected through a digital input. The collected data is stored and sent to a message queue for use by other tasks.

The control task processes sensor data and determines system behavior. The LED is activated when motion is detected, and the servo motor (fan) speed is adjusted based on temperature thresholds. The fan operates corresponding to temperature ranges, ensuring simple and predictable control.

The display task retrieves sensor data from the queue and updates the LCD. A mutex is used to ensure exclusive access to the display. The display shows temperature, motion status, light state, fan speed, and system status.

The system uses inter-task communication mechanisms including queues, semaphores, and mutexes to ensure synchronization and to avoid unwanted results. Our design allows tasks to operate independently while maintaining coordinated system behavior.

Wiring and Hardware Method

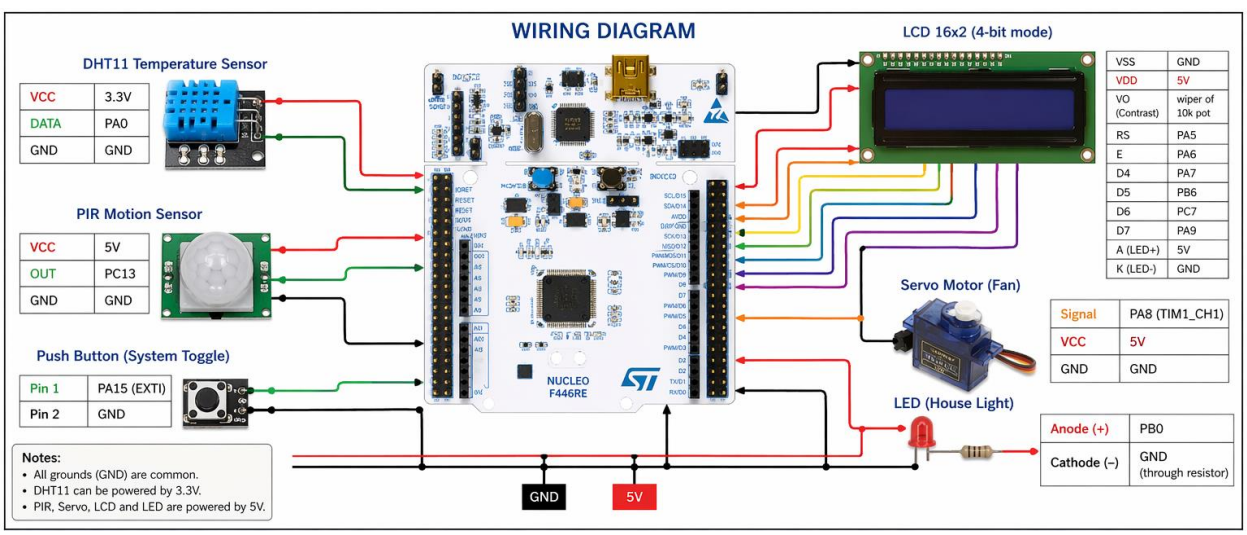


Fig. 1 Wiring diagram for smart home system

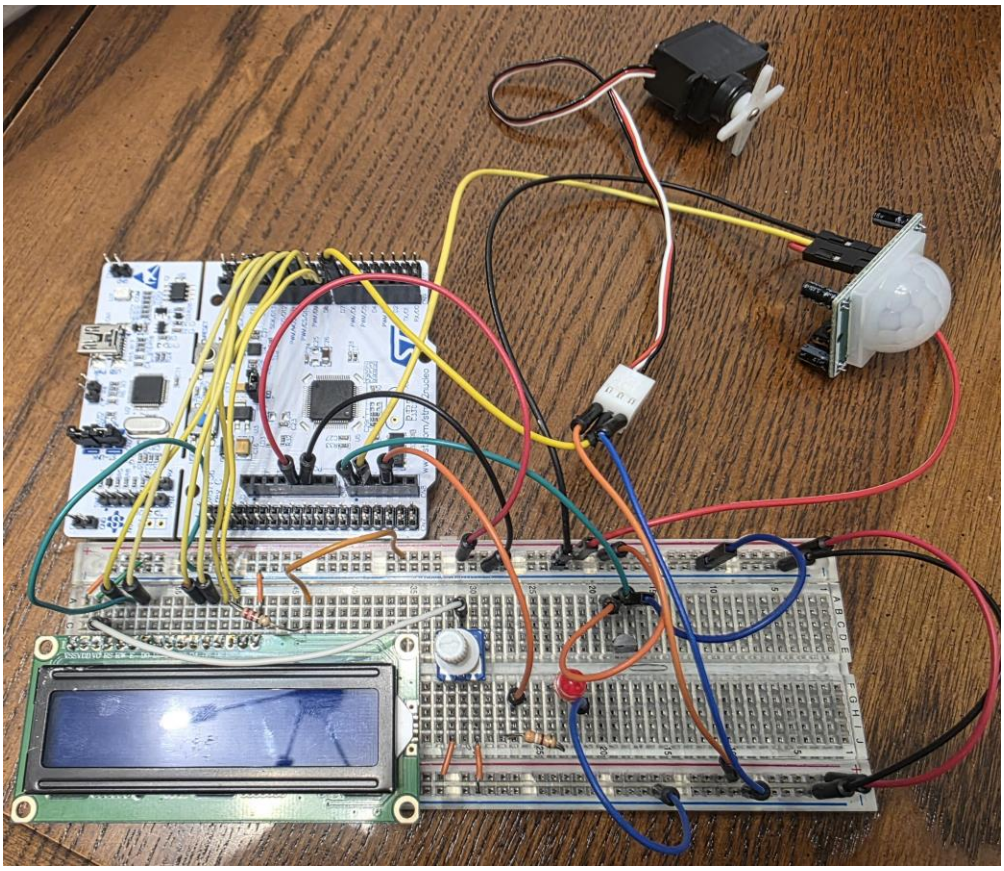
The hardware components were all connected to the STM32 NUCLEO-F446RE. The temperature sensor is connected to an analog input pin (ADC Channel 0). The sensor outputs a voltage proportional to temperature, which is read by the ADC. The PIR motion sensor is

connected to a digital input pin configured with interrupt capability. The sensor outputs a HIGH signal when motion is detected. The LED is connected to a GPIO output pin. The LED is controlled by the system to represent lighting based on motion detection. The servo motor acting as our fan is connected to a PWM-capable pin. The servo position is controlled by varying the PWM duty cycle, simulating fan speed levels. The LCD display is connected using multiple GPIO pins for RS, Enable, and data lines. The display runs in 4-bit mode to reduce pin usage. All components share the ground and appropriate power supply connections.

Summary of Method

Overall, the system integrates hardware and software using a real-time multitasking approach. Peripheral initialization makes sure that our hardware runs properly in conjunction with the code, while FreeRTOS enables structured task execution. The use of queues, mutexes, and semaphores provides reliable communication and synchronization.

Results & Discussion



The smart home controller system was successfully implemented and tested using the STM32 NUCLEO-F446RE board with FreeRTOS, demonstrating reliable real-time performance across

all integrated components. The system operated using a multitasking architecture in which sensing, control, display, and user input functions were executed concurrently without noticeable delay. Sensor data was consistently collected and transmitted through the message queue, allowing seamless communication between tasks and ensuring that system responses were both timely and accurate.

The temperature-based control of the fan, represented by a servo motor, functioned as designed. Temperature readings obtained through the ADC were converted into Fahrenheit and mapped to control levels. As the temperature increased, the servo motor adjusted its position to simulate increasing fan speed, with smooth and stable transitions observed between each operating level. The PWM signal generated by Timer 1 provided precise control, and the fan responded to changes in environmental conditions.

The motion detection subsystem also performed effectively, with the PIR sensor accurately detecting movement and triggering the LED output in real time. When motion was detected, the LED turned on immediately, and when motion ceased, the LED turned off. This behavior confirmed that the system was capable of handling event-driven inputs efficiently.

The LCD display provided clear feedback on system status, including temperature, motion detection, light state, fan speed, and overall system operation. The display updated reliably without flickering or data corruption, which verified that the mutex-based synchronization method successfully prevented conflicts between tasks attempting to access the shared display resource.

In conclusion, our final configuration of our system remained stable throughout final testing, with no observed crashes or unexpected behavior. FreeRTOS features, including queues, semaphores, mutexes, and timers, operate correctly and enable coordination between tasks. The results confirm that the smart home controller effectively meets its design objectives by providing real-time monitoring, responsive control, and reliable operation within a multitasking embedded system environment.

Conclusions

In this project, a smart home system was successfully designed and implemented using the STM32 NUCLEO-F446RE microcontroller and FreeRTOS. The system demonstrated the ability to monitor environmental conditions and control multiple devices in real time through a structured multitasking approach. By assigning dedicated tasks to sensing, control, display, and user interaction, the system achieved efficient and organized operation while maintaining responsiveness and stability.

The integration of key FreeRTOS features, including queues, mutexes, and semaphores, enabled reliable communication and synchronization between tasks. Sensor data was accurately collected and processed, allowing the system to make real-time decisions such as adjusting fan speed based on temperature and activating lighting in response to motion. The LCD display provided feedback to the user.

The results confirm that using a real-time operating system significantly improves the performance and scalability of embedded systems compared to traditional single-loop designs. The system operated consistently without errors, demonstrating success with both hardware and software integration. This project shows the effectiveness of combining embedded systems with RTOS-based design to develop smart, automated solutions.

Link to Github repo: SpaceCadet2121/Embedded_Final_Project

Acknowledgements

References

[1] Analog Devices, "Low Voltage Temperature Sensors PIN CONFIGURATIONS 1 2 3 5 4 TOP VIEW (Not to Scale) NC = NO CONNECT V OUT SHUTDOWN GND NC +V S Figure 2. RJ-5 (SOT-23)." Available: https://www.analog.com/media/en/technical-documentation/data-sheets/TMP35_36_37.pdf

[2] A. KS, "16x2 LCD Pinout Diagram | Interfacing 16x2 LCD with Arduino," *Electronics For You*, Jul. 31, 2020. https://www.electronicsforu.com/technology-trends/learn-electronics/16x2-lcd-pinout-diagram

[3] Components 101, "HC-SR501 PIR Sensor Working, Pinout & Datasheet," *Components101*, Jul. 18, 2021. https://components101.com/sensors/hc-sr501-pir-sensor

[4] Arduino, "Servo Motor Basics with Arduino," *docs.arduino.cc*, Mar. 04, 2024. https://docs.arduino.cc/learn/electronics/servo-motors/

[5] "FreeRTOS with CubeMX." Accessed: Apr. 21, 2026. [Online]. Available: https://blog.brichacek.net/wp-content/uploads/2015/10/STM32F4-and-FreeRTOS.pdf